Distributed Tracing

chukmunnlee@nus.edu.sg

© Copyright National University of Singapore

What is (Distributed) Tracing?

- Traces a request as it traverses through an application
  - Either monolithic, microservices or serverless
  - Provides end-to-end visibility especially in micro services
  - Captures the interaction between services

From OTel Demo application - https://github.com/open-telemetry/opentelemetry-demo

© Copyright National University of Singapore

Why Distributed Tracing?

- Reason for tracing
  - Visibility across services boundaries
    - Traces a single request as it traverses the application
  - Root cause analysis
    - Identifies where an error has occurred in a service request call/chain
  - Understand inter dependencies between services
    - Reveal the relationship between services, helps with debugging and making architectural decisions
  - Identify bottlenecks
    - Know exactly where time is spent in the system
- Helps answers the following types of questions
  - Where did a failure occur?
  - Why was the request slow?
  - Where is the most vulnerable/weakest part of the system?

From OTel Demo application - https://github.com/open-telemetry/opentelemetry-demo

© Copyright National University of Singapore

Example - View of a Single Trace

Trace Start **August 25 2025, 10:33:53.064** | Duration **110.7ms** | Services **13** | Depth **15** | Total Spans **59**

Service & Operation	Operation	Duration
load-generator	POST	
frontend-proxy	ingress	
frontend-proxy	router frontend egress	
frontend	POST	106.85ms
frontend	POST /api/checkout	106.11ms
frontend	executing api route (pages) /api/checkout	105.58ms
frontend	grpc.oteldemo.CheckoutService/PlaceOrder	87.01ms
checkout	oteldemo.CheckoutService/PlaceOrder	85.91ms
checkout	prepareOrderItemsAndShippingQuoteFromCart	68.32ms
checkout	oteldemo.CartService/GetCart	11.71ms
cart	POST /oteldemo.CartService/GetCart	11.21ms
cart	HGET	2.99ms
checkout	oteldemo.ProductCatalogService/GetProduct	3.04ms
product-catalog	oteldemo.ProductCatalogService/GetPr...	105µs
checkout	oteldemo.CurrencyService/Convert	10.38ms
currency	Currency/Convert	3.42ms
checkout	oteldemo.ProductCatalogService/GetProduct	2.31ms
product-catalog	oteldemo.ProductCatalogService/GetPr...	79µs
checkout	oteldemo.CurrencyService/Convert	6.18ms
currency	Currency/Convert	3.13ms
checkout	oteldemo.ProductCatalogService/GetProduct	7.31ms
product-catalog	oteldemo.ProductCatalogService/GetPr...	157µs
checkout	oteldemo.CurrencyService/Convert	8.7ms
currency	Currency/Convert	1.06ms
checkout	oteldemo.ShippingService/GetQuote	10.53ms
shipping	oteldemo.ShippingService/GetQuote	7.75ms
shipping	POST	7.67ms
quote	POST /getquote	234µs
quote	{closure}	140µs
quote	calculate-quote	23µs
checkout	oteldemo.CurrencyService/Convert	7.84ms
currency	Currency/Convert	4.26ms

© Copyright National University of Singapore

Example - Trace Structure

Unique trace id

Part of the trace

Unique span id

Child of the following span

Spans in the trace

Span attributes

Executed by this process


```
{
  "traceID": "bcbd51a22dead3b11f5db71c65f212ae",
  "spans": [
    {
      "traceID": "bcbd51a22dead3b11f5db71c65f212ae",
      "spanID": "6b4979e37749a8aa",
      "operationName": "middleware - \u003canonymous\u003e",
      "references": [ { "refType": "CHILD_OF", "traceID": "bcbd51a22dead3b11f5db71c65f212ae", "spanID": "245c54d949be60c5" } ],
      "startTime": 1755824090060000,
      "duration": 23,
      "tags": [ { "key": "express.name", "type": "string", "value": "\u003canonymous\u003e" }, ... ],
      "processID": "p1", ...
    },
    {
      "traceID": "bcbd51a22dead3b11f5db71c65f212ae",
      "spanID": "d8ce7d5714f50ab0",
      "operationName": "request handler - /api/book/:isbn13",
      "references": [ { "refType": "CHILD_OF", "traceID": "bcbd51a22dead3b11f5db71c65f212ae", "spanID": " 6b4979e37749a8aa" } ],
      "startTime": 1755824090060000,
      "duration": 906, ...
  ],
  "processes": {
    "p1": {
      "serviceName": "books",
      "tags": [ { "key": "process.command_args", "type": "string",
          "value": "[\"/usr/bin/node\",\"--require\",\"./src/instrument.js\",\"/home/cmlee/src/soda/cno/books/src/main.js\"]" },
        { "key": "process.pid", "type": "int64", "value": 1889222 },
      ....

  }
```

Core Concepts

- Trace captures the entire journey of a request through an application
  - Traces are identified by their unique trace id
- Span are individual operations within a trace, a single unit of work
  - Within a process, between processes eg. authenticate user, perform database query
  - Spans are identified by their unique span id
  - Show the order of operations in the form in hierarchical structure or parent-child relationship
  - Spans record duration, collectively forms the latency of a request
- Span context is the metadata structures that captures the relationship between a trace and its spans and between spans
  - Includes other contextual information eg. trace flags, baggage
  - Propagated between services through HTTP headers or gRPC metadata
- Tags/labels/attributes are attached to traces and spans
  - Eg. `user.id`, `http.method`, `db.statement`
  - Use to filter spans or locate spans

© Copyright National University of Singapore

Span

- Span represents a single unit of operation in a trace
- Trace consist of one or more related spans
  - Each span defines the work that is performed as part of the trace
- Span relationships
  - Parent - child - parent span created the child span
    - Eg. checkout span created the payment span
  - Siblings - spans that are created by the same parent span
    - Eg. checkout span creates payment and create delivery order spans; payment and create delivery order spans are siblings
- Every span has a parent span except the root span
- Trace to span and span to span relationships are captured in specific attributes of a span

© Copyright National University of Singapore

Anatomy of a Span

- Intrinsic information
  - Identifiers
    - traceID - links a span to a trace
    - spanID - span's unique identifier
    - parentSpanID - parent spanID, null if root
  - Timing information
    - Span duration, start/end time
  - Operation details
    - operation name
    - type of span - CLIENT, SERVER, INTERNAL
    - status - if the operation is successful
- Events describes what happens during a span execution
  - Eg. inventory count, exchange rate
  - Does not affect the span's status
- Exception record error conditions and failures
  - Spans are typically marked as ERROR
- Standard and operation specific attributes
  - Standard - depending on the type of operation
    - Eg. Service - service.name, service.version
    - Eg. HTTP request - http.method, http.status_code, http.url
    - Eg. database - db.name, db.statement
  - Custom - any key value pair
- Links establishes relationships between a span and some entity that may not be part of a trace
  - Eg. span starting a batch job, a link can be used to record the batch ID

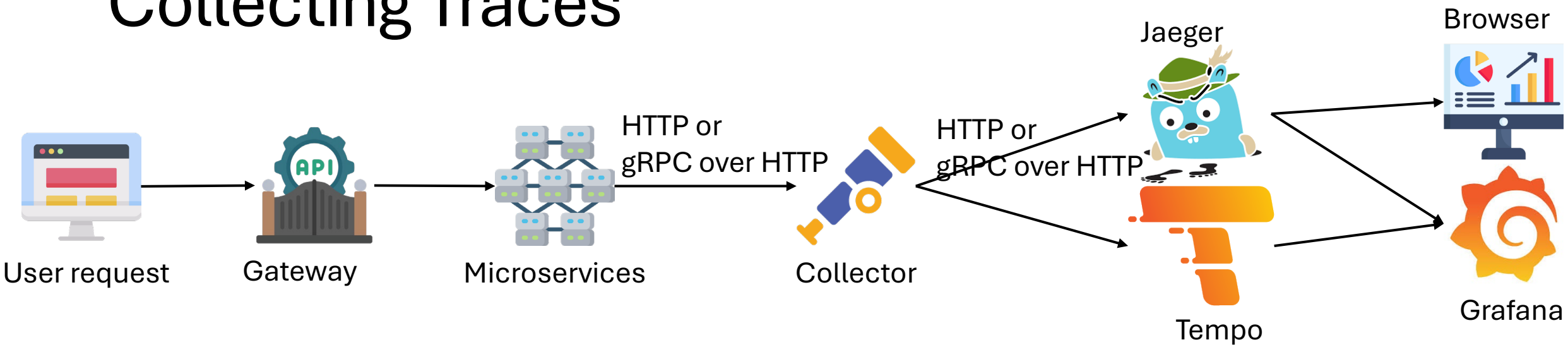
© Copyright National University of Singapore

Span Context

- Context keeps track to metadata, allowing trace and spans to be correctly correlated
  - What is the current trace?
  - What is the current active span?
- Context has to be propagated across process boundaries in order for down stream services spans to be created correctly
- Active context is the context that will populate the **`traceID`** and **`spanParentID`** when a span is created
  - Spans can create new context but share the same **`traceID`**

© Copyright National University of Singapore

Collecting Traces



- A trace is created by the first component that receives the request
  - Typically, the ingress point eg. load balancer, API gateway, etc
  - Generates a trace id
- As the request traverse the application, every service will generate create a span to record its operation
- Traces are exported to a collector to be processed and enriched
  - Eg. correlating traces with metrics
- Processed traces are forwarded to tracing backend for visualization, querying and storage

© Copyright National University of Singapore

Auto Instrumentation

© Copyright National University of Singapore

Instrumentation

- Instrumentation is the process of generating telemetry signals from an application
  - Emit the following telemetry data: traces, logs and metrics
- Two approaches to instrumentation
  - Auto - emit telemetry data with little or no code changes to an application
    - Work by installing agents, inserting additional code at runtime
    - Supported in popular frameworks like Flask, SpringBoot, express, Gin, etc
    - Easy to start, only supports instrumentation at the framework level
  - Manual - explicitly add code at specific point in the application to emit telemetry data
    - Eg. At the start of a request handler
    - Emit fine grained telemetry data with rich application specific context
    - Require code changes

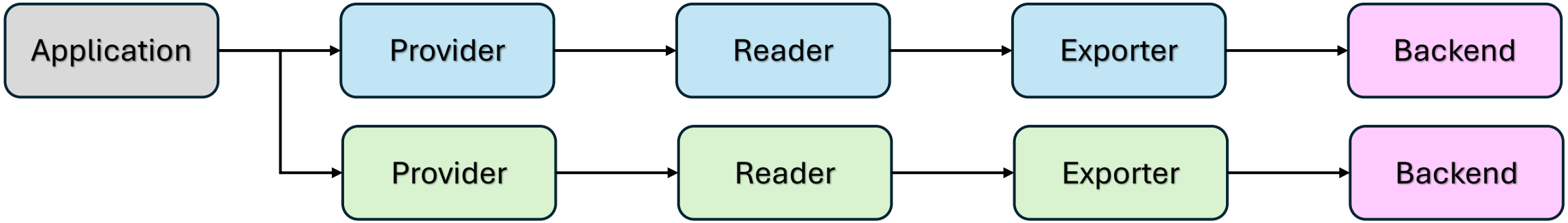
© Copyright National University of Singapore

Open Telemetry

- Open Telemetry is an open-source observability framework
  - Designed to provide a standardized way to instrument, collect and export telemetry data
  - Works with logs, traces and metrics
- Open Telemetry consist of
  - Specification - https://opentelemetry.io/docs/specs/
  - Collector - https://opentelemetry.io/docs/collector
    - A vendor-agnostic implementation agent for collecting, processing and exporting telemetry data
  - SDK - https://opentelemetry.io/docs/languages/
    - Support for popular languages like Java, Go, JavaScript, Python
    - JavaScript reference - https://open-telemetry.github.io/opentelemetry-js/

© Copyright National University of Singapore

OpenTelemetry SDK Concepts



- OpenTelemetry is modular, catering to every conceivable configuration
- Core components
  - Provider creates and manage telemetry objects like traces, metrics
    - Eg. **`TraceProvider`**
  - Reader collates, batches and aggregates data from an upstream provider
    - Eg. **`BatchSpanProvider`**
  - Exporter receives batches of data from readers and forwards them to a backend for processing
    - Eg. **`ConsoleSpanExporter`**, **`OTLPTraceExporter`**

© Copyright National University of Singapore

Auto Instrumenting Traces with OTel JavaScript SDK

- Two ways to auto instrument JavaScript application
  - Zero-Code - instrument the application without adding any code
    - Eg. `java -javaagent:./opentelemetry-javaagent.jar -jar app.jar`
    - Eg. `opentelemetry-instrument python main.py`
  - Minimal code - code to setup the OTel SDK to perform instrumentation
    - Serve as bootstrap for full manual instrumentation
    - Eg. JavaScript see https://opentelemetry.io/docs/zero-code/js/
- Install the OTel SDK for minimal code auto instrumentation
  - `@opentelemetry/sdk-node`
  - `@opentelemetry/api`
  - `@opentelemetry/auto-instrumentations-node`

© Copyright National University of Singapore

Bootstrap Auto Tracing

```
const otlpTraceExporter = new OTLPTraceExporter({
  url: 'http://otel-collector:4317'
})
const consoleTraceExporter = new ConsoleSpanExporter()

const batchToOTLP = new BatchSpanProcessor(otlpTraceExporter)
const toConsole = new SimpleSpanProcessor(consoleTraceExporter)

const sdk = new NodeSDK({
  resource: resourceFromAttributes({
    [ATTR_SERVICE_NAME]: package.name,
    [ATTR_SERVICE_VERSION]: package.version
  }),
  spanProcessors: [ batchToOTLP, toConsole ],
  instrumentations: getNodeAutoInstrumentations(),
})

sdk.start()
```

Traces are exported using gRPC over HTTP to the backend specified by url

Traces are batched, for efficiency, before exporting

Convenience class to initialize the trace provider and link providers to readers

Sets the trace attribute

Registers instrumentation plugins for popular frameworks

Initializes and starts the instrumentation

© Copyright National University of Singapore

Starting Applications with Auto Instrumentation

NodeJS specific

```
node --require instrument.js main.js
```

- Certain runtime like Node require the auto instrumentation code to be loaded before executing the main program
- **`--require`** instruct node to load **`instrument.js`** first before **`main.js`**
- OTel SDK requires auto instrumentation
  - Patching the frameworks like Express, MySQL, HTTP etc. modules so that the correct signals, eg traces, can be emitted
  - Otherwise auto instrumentation will not work

© Copyright National University of Singapore

Semantic Convention

- Standardized naming convention for key/value pairs
  - Trace tags, metrics labels, etc.
  - Eg. **`service.name`**, **`service.version`**, **`http.method`**, **`http.status_code`**, **`trace_id`**
- Important because
  - Use the same key across services and telemetry data to mean the same thing
    - Eg. **`service.name`** vs **`service_name`**, **`serviceName`**
  - Allow different telemetry data from different services to be aggregated and analysed by different backends
    - Eg. Easy to query and filter if the keys across different data are the same and name the same thing
- OTel SDK provides a set of constants in **`@opentelemetry/semantic-conventions`** package
  - Eg. **`ATTR_HTTP_REQUEST_METHOD`**, **`ATTR_SERVICE_NAME`**, etc.

© Copyright National University of Singapore

Workshop 1
Task 1

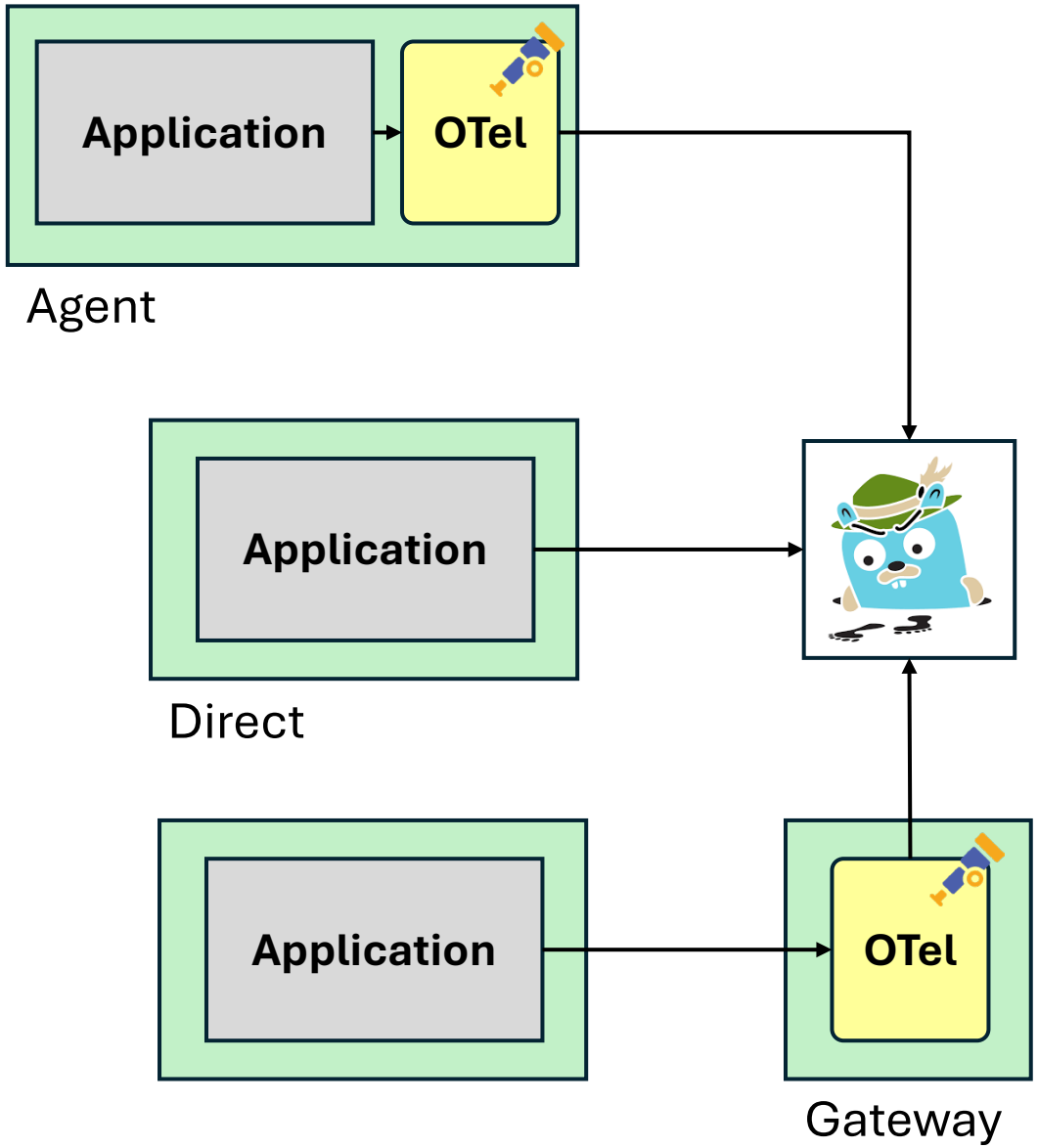
© Copyright National University of Singapore

Open Telemetry Collector

© Copyright National University of Singapore

Open Telemetry Collector

- Component that receives telemetry data from applications and forwards them to backends
- Deployed as
  - Agent - the collector runs in the same host as the application, like a sidecar
  - Gateway - centralised service for telemetry for aggregating data from multiple sources and sending exporting to multiple backends
- Benefit of proxying thru OTel collector vs send data directly to backends
  - Decouple application from a specific backend; flexibility to change backend without affecting the application
  - Enriching and cleaning the telemetry data eg. add region labels, sampling to reduce volume, etc
  - Consolidate, reduce and simplify telemetry data configuration



© Copyright National University of Singapore

Main Concepts

- Pipeline defines how telemetry data is ingested, processed and forwarded as it flows through the collector
- Core components
  - Receivers are data ingestion points
    - Ports to listen on and the protocol
    - Common protocols include HTTP and gRPC over HTTP
  - Processors is where the data are inspected, modified or enriched
    - Examples of processors include batching, sampling, dropping/adding labels
    - Processors are optional
  - Exporters send data to their respective backends
    - Define the endpoint the protocol to use
    - Eg. metrics are exported to Prometheus or Mimir over HTTP
- Pipelines are defined in a YAML file and passed to OTel collector when it starts up

© Copyright National University of Singapore

Configuration

Image from https://opentelemetry.io/docs/collector/

© Copyright National University of Singapore

Open Telemetry Collector Configuration

- YAML file with the following sections
  - receivers - how data are collected
    - https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/receiver
  - processors - how data are processed
    - https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/processor
  - exporters - where data are forwarded
    - https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/exporter
  - service - uses receivers, processors and exporters to define a processing pipeline
  - connectors - connecting data flow from one pipeline to a different pipeline, optional
    - https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/connector
  - extensions - additional functionalities like health checks, authentication, etc. optional
    - https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/extension

© Copyright National University of Singapore

Telemetry Processing Pipeline

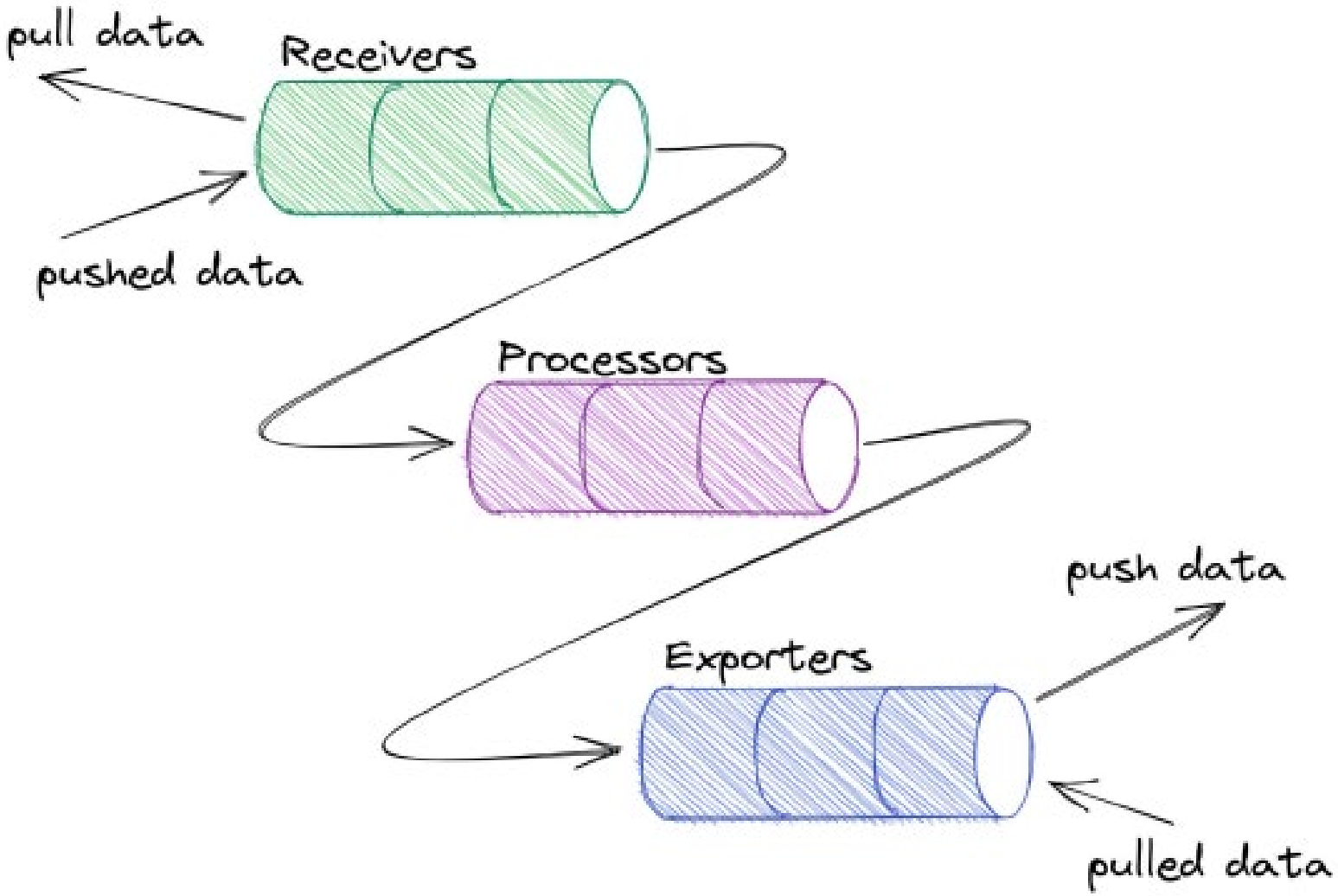


Image from https://medium.com/@siddhuajay001/mastering-the-opentelemetry-collector-architecture-deployment-and-core-components-3ebe7e1c9c39

© Copyright National University of Singapore

Example - OTel Collector Configuration - 1

OTLP protocol receiver; opening 2 ports for gRPC and HTTP

See https://github.com/open-telemetry/opentelemetry-collector/tree/main/receiver/otlpreceiver

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318
```

The endpoints for receiving trace and other signals

2 OTLP based exporters; debug sends the output to console

```
exporters:
  debug:
    verbosity: normal
  otlp/jaeger:
    endpoint: jaeger:4317
    tls:
      insecure: true
  otlp/tempo:
    endpoint: tempo:4317
    tls:
      insecure: true
    sending_queue:
      queue_size: 5000
```

```
processors:
  batch:
  memory_limiter:
    limit_mib: 2048
    spike_limit_mib: 512
```

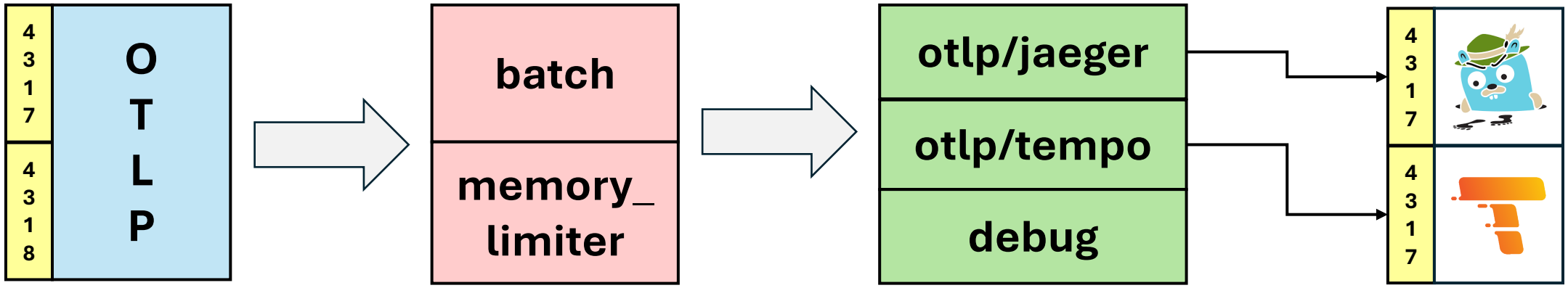
Configure 2 processors: batching limiting the memory

© Copyright National University of Singapore

Example - OTel Collector Configuration - 2

Assemble a pipeline to process traces

```
service:
  pipelines:
    traces:
      receivers: [ otlp ]
      processors: [ batch, memory_limiter ]
      exporters: [ otlp/jaeger, otlp/tempo, debug ]
```



Example - OTel Collector Configuration

- OTLPTraceExporter exports traces using either HTTP or gRPC protocol
  - HTTP - **`@opentelemetry/exporter-trace-otlp-http`**
    - To 4318 port
  - gRPC - **`@opentelemetry/exporter-trace-otlp-grpc`**
    - To 4317 port

```
const { OTLPTraceExporter } = require('@opentelemetry/exporter-trace-otlp-http')
```

Using HTTP transport

```
const otlpTraceExporter = new OTLPTraceExporter(
   {url: 'http://otel-collector:4318'})
```

Export to OTel collector HTTP endpoint

© Copyright National University of Singapore

Common Record Structure

- Strength of OTel is that traces, logs and metrics share a consistent record structure
  - Common attribute labels and data types, allows correlation across the signal
    - Eg. service name, time format
  - Simplify application instrumentation
- Record structure
  - Resource - information on the service that produces the signal
    - Eg. service name, service version, host name, namespace, etc
  - Attributes - key/value pair for storing contextual data about the specific signal,
    - Eg. error message, latency of a request, etc
  - Instrumentation - identifies the library/agent that is responsible for generating the signal
  - Signal specific parts

© Copyright National University of Singapore

Trace Record Structure

- Resource - metadata about the entity producing the trace
  - **`service.name`**
  - **`service.version`**
  - Other service metadata like host name, deployment environment, etc
- Span attributes - metadata about the specific unit of work
  - **`trace_id`** - a unique identifier for the entire trace
  - **`span_id`** - a unique identifier for an operation within a trace
  - **`parent_span_id`** - the span id of the parent operation that created this span
  - **`name`** - description of the operation, human friendly
  - **`kind`** - role of the span eg. server, client, internal, producer, etc
  - **`start_time_unix_nano`**, **`end_time_unix_nano`** - starting and ending time of the operation
  - **`attributes`** - collection of key/value pairs providing contextual data for the span
    - Eg. http.method, http.status_code, db.statement, etc

© Copyright National University of Singapore

Example - OTLP Trace Record

```
Resource attributes:
     -> host.name: Str(batcomputer)
     -> host.arch: Str(amd64)
     -> host.id: Str(b93676df0cfa42289ae466b615b58c1b)
     -> process.pid: Int(412411)
     -> process.executable.name: Str(node)
     -> process.executable.path: Str(/usr/bin/node)
     -> process.command_args: Slice(["/usr/bin/node","--require","./src/instrument.js","/home/cmlee/src/soda/cno/solutions/day03/books/src/main.js"])
     -> process.runtime.version: Str(24.0.1)
     -> process.runtime.name: Str(nodejs)
     -> process.runtime.description: Str(Node.js)
     -> process.command: Str(/home/cmlee/src/soda/cno/solutions/day03/books/src/main.js)
     -> process.owner: Str(cmlee)
     -> service.name: Str(books)
     -> service.version: Str(1.0.0)
ScopeSpans #0
ScopeSpans SchemaURL:
InstrumentationScope @opentelemetry/instrumentation-express 0.52.0
Span #0
    Trace ID       : b2fce9151be9f95ef8d6a2a3aecb729b
    Parent ID      : 3e6b57a0062e77ed
    ID             : 04876ca5681caf84
    Name           : router - /api
    Kind           : Internal
    Start time     : 2025-10-24 05:11:35.468 +0000 UTC
    End time       : 2025-10-24 05:11:35.468095702 +0000 UTC
    Status code    : Unset
    Status message :
Attributes:
     -> http.route: Str(/api)
     -> express.name: Str(/api)
     -> express.type: Str(router)
Span #1
    Trace ID       : b2fce9151be9f95ef8d6a2a3aecb729b
    Parent ID      : 3e6b57a0062e77ed
    ID             : 707d0e39eef777ed
    Name           : middleware - <anonymous>
    Kind           : Internal
    Start time     : 2025-10-24 05:11:35.468 +0000 UTC
    End time       : 2025-10-24 05:11:35.47249654 +0000 UTC
```

© Copyright National University of Singapore

Workshop 1
Task 2

© Copyright National University of Singapore

Working with Traces

© Copyright National University of Singapore

Visualizing Tools

- Jaeger and Tempo are open-source end-to-end distributed tracing system
  - Helps monitor and troubleshoot interactions between services
  - Jaeger - https://github.com/jaegertracing/jaeger
  - Tempo - https://github.com/grafana/tempo
- Differences
  - Jaeger - https://www.jaegertracing.io/docs/next-release/getting-started/
    - Build in UI allow users to visualize and query traces and the application architecture
      - Accessed on port 16686
    - Can be integrated with Grafana
  - Tempo
    - More complex to setup, see https://grafana.com/docs/tempo/latest/configuration/
    - No UI, require a visualizer like Grafana
    - Powerful query language call TraceQL
- Both uses port 4317 and 4318 for gRPC and HTTP traffic respectively

© Copyright National University of Singapore

Jaeger

- Allow you to search traces based on service, operations and tags
  - Filters must us logfmt eg **`app.cache_hit=true`** as predicate
- Features
  - Visualize the all the discovered services in the application, shows relationships between them
  - Visualize traces and spans within them
    - Spans are shown in hierarchy, reveal how a request is processed by services
    - Critical path shows how long each spans takes to complete its task
    - Plot flame graph to show where a trace is spending time, viz. identifying major latency contributor

© Copyright National University of Singapore

Jaeger - Main Interface

JAEGER UI | Search | Compare | System Architecture | Monitor | Lookup by Trace ID... | About Jaeger

Search | Upload

Service (18)
recommendation

Operation (4)
all

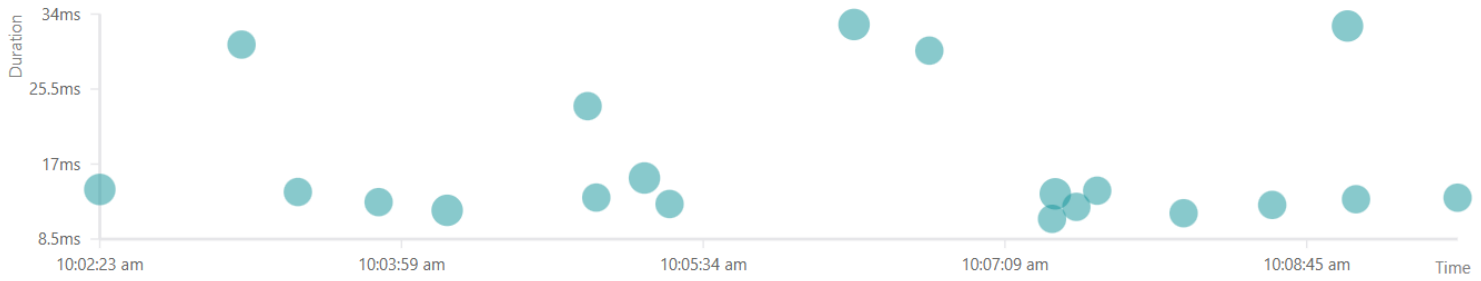
Tags
http.status_code=200 error=true

Lookback
Last 15 Minutes

Max Duration: e.g. 1.2s, 100ms, ...
Min Duration: e.g. 1.2s, 100ms, ...

Limit Results
20

Find Traces



20 Traces | Sort: Most Recent | Download Results | Deep Dependency Graph

Compare traces by selecting result items

Trace search results. Can be filtered by tags in logfmt format

load-generator: user_get_recommendations 906675b — 13.19ms
18 Spans | frontend (8) | frontend-proxy (2) | load-generator (2) | product-catalog (4) | recommendation (2) | Today 10:09:32 am, a few seconds ago

load-generator: user_get_recommendations 6e5f0c2 — 13.02ms
18 Spans | frontend (8) | frontend-proxy (2) | load-generator (2) | product-catalog (4) | recommendation (2) | Today 10:09:00 am, a few seconds ago

File Explorer

© Copyright National University of Singapore

Jaeger - Trace View

JAEGER UI | Search | Compare | System Architecture | Monitor | Lookup by Trace ID... | About Jaeger

load-generator: user_get_recommendations
906675b

Find... | Trace Timeline

Trace Start **August 28 2025, 10:09:32.979** | Duration **13.19ms** | Services **5** | Depth **10** | Total Spans **18**

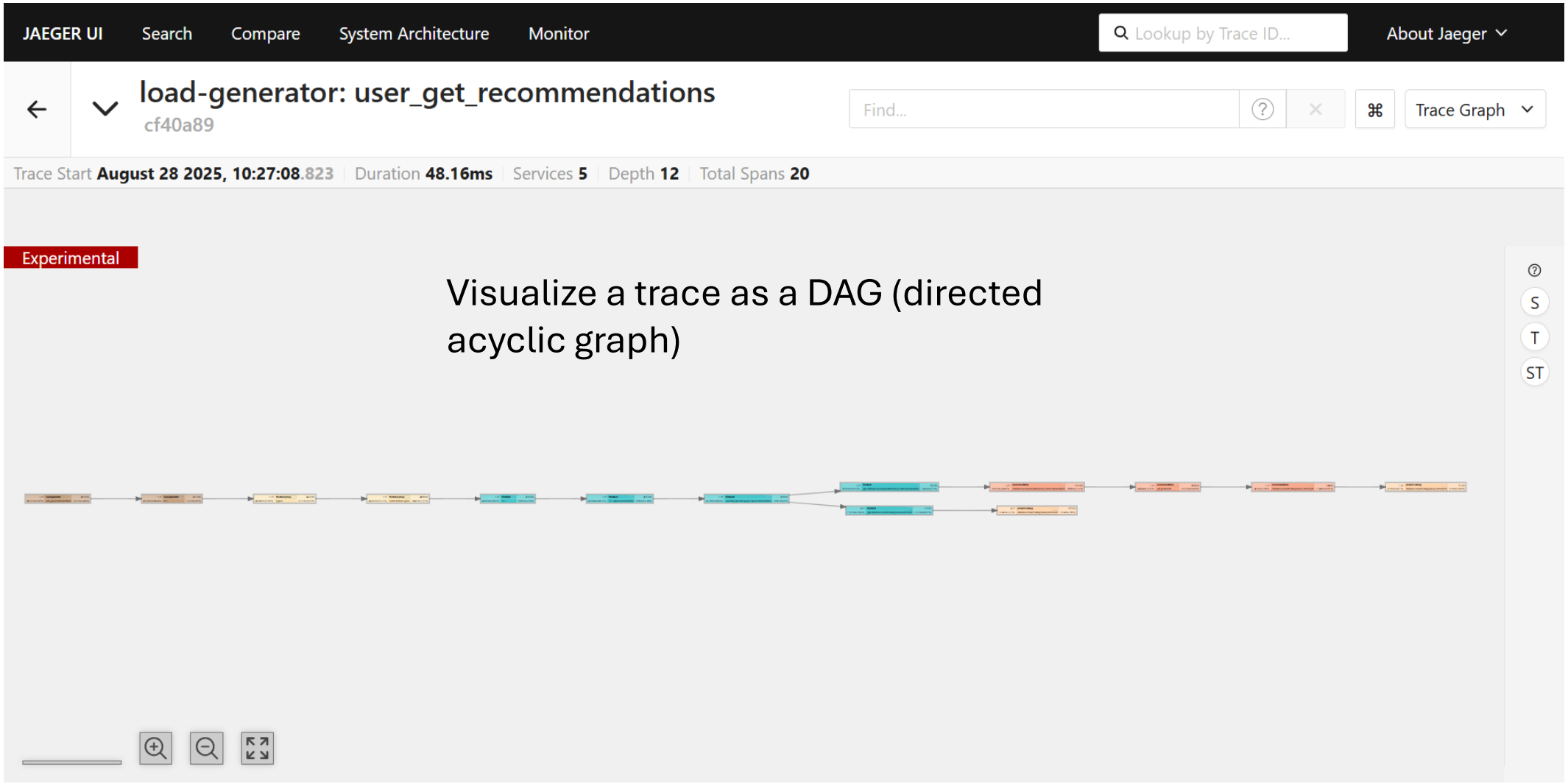
0µs | 3.3ms | 6.6ms | 9.89ms | 13.19ms

Service & Operation	Span	Duration
load-generator	user_get_recommendations	
load-generator	GET	12.29ms
frontend-proxy	ingress	10.52ms
frontend-proxy	router frontend egress	10.33ms
frontend	GET	9.66ms
frontend	GET /api/recommendat...	8.96ms
frontend	executing api rout...	8.11ms
frontend	grpc.oteldem...	3.61ms
recommendatio...		698µs
recommenda...		375µs
frontend	grpc.oteldem...	3.53ms
product-catalog ...		581µs
frontend	grpc.oteldem...	3.36ms
product-catalog ...		361µs
frontend	grpc.oteldem...	frontend::grpc.oteldemo.ProductCatalogService/GetProduct | 3.4ms

Spans and critical paths in a trace.

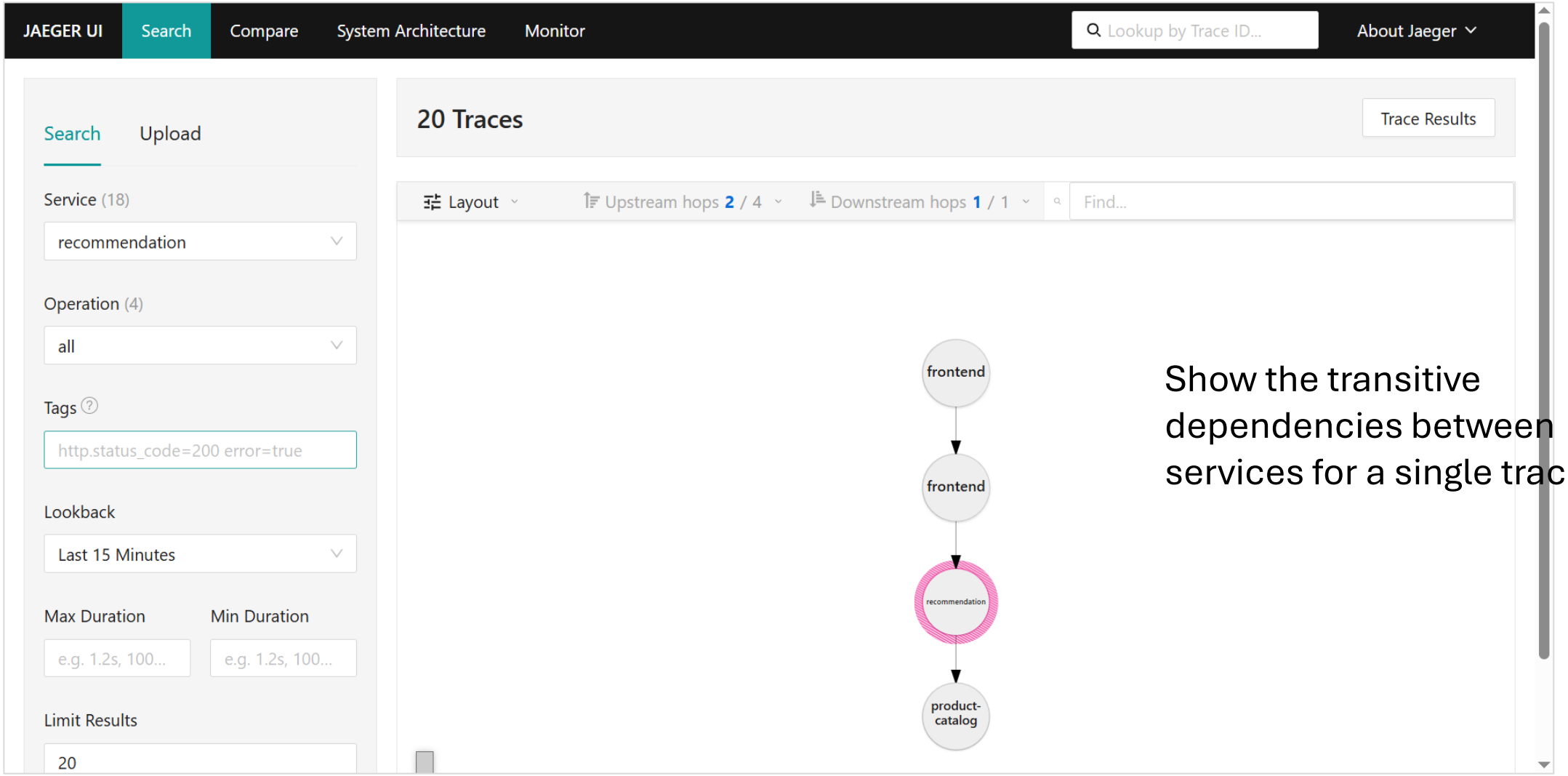
Includes duration of each span

Jaeger - Trace Graph



Visualize a trace as a DAG (directed acyclic graph)

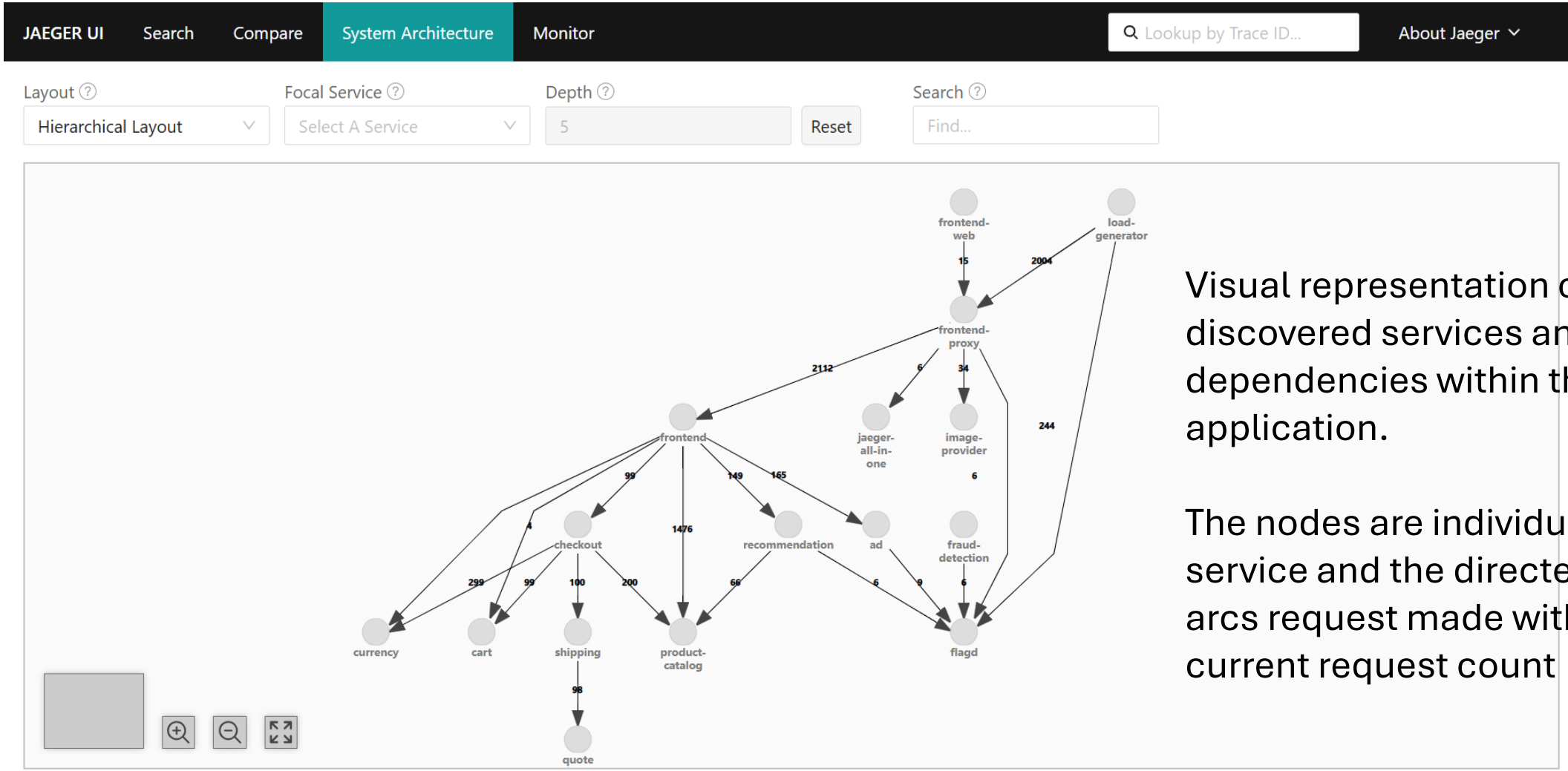
Jaeger - Deep Dependency Graph



Show the transitive dependencies between services for a single trace

© Copyright National University of Singapore

Jaeger - System Architecture



Visual representation of the discovered services and its dependencies within the application.

The nodes are individual service and the directed arcs request made with the current request count

Workshop 1
Task 3

© Copyright National University of Singapore

Grafana Tempo

- Distributed tracking backend like Jaeger
  - Developed by Grafana, seamlessly integrates into Grafana ecosystem
- Does not have a UI, relies on Grafana for visualization
  - Lightweight and efficient but more complex to setup
  - Supports TraceQL, a powerful querying language
- Grafana Tempo requires a configuration file to start
  - Configuration file is very similar to OTel Collector configuration file
  - https://grafana.com/docs/tempo/latest/configuration/
  - See provided sources for example of Tempo configuration

```
tempo -config.file=/path/to/temp-config.yaml
```

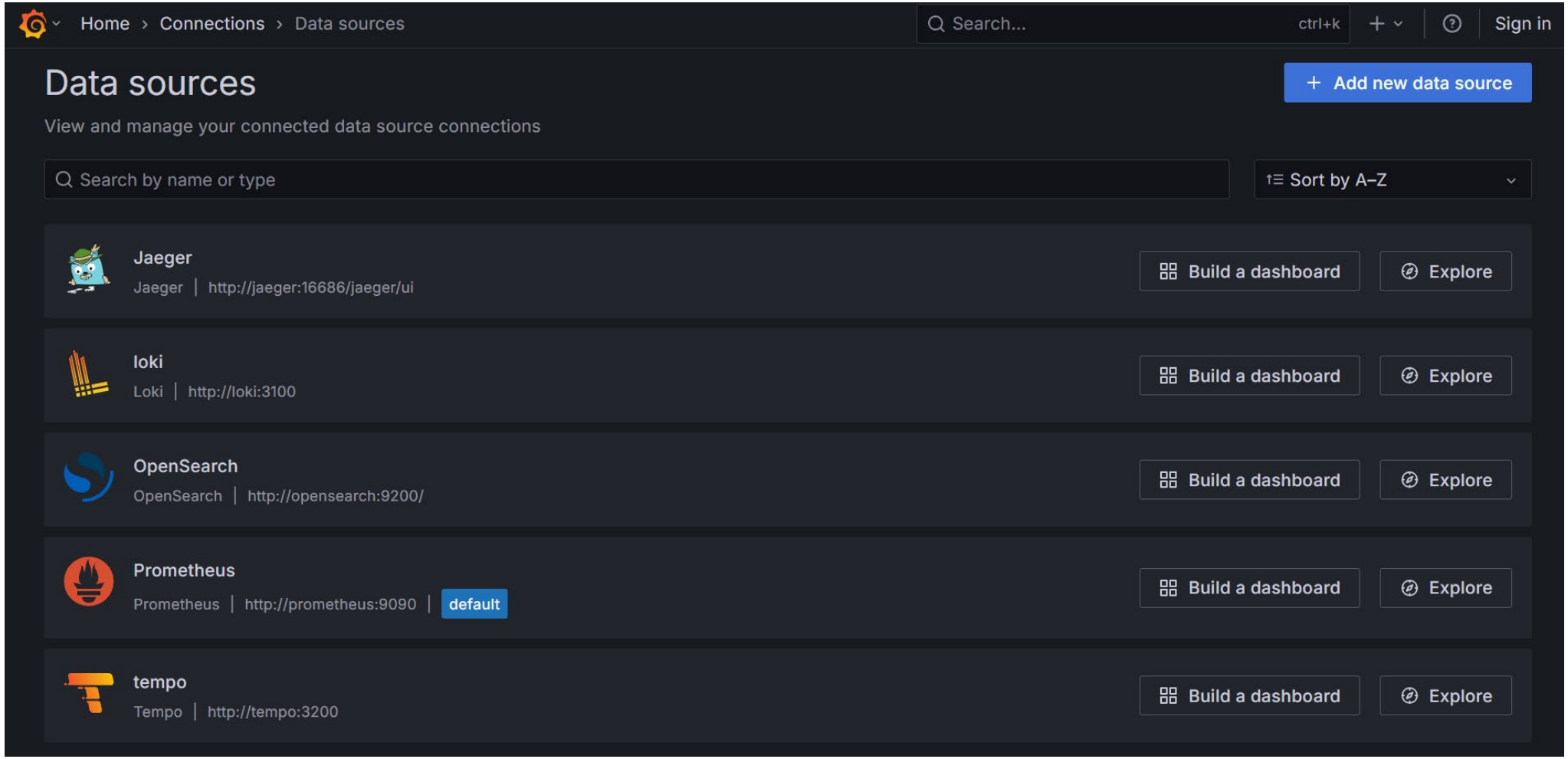
© Copyright National University of Singapore

Grafana

- Grafana is a popular open-source platform form monitoring, visualizing and analyzing data
- Key features
  - Creating different types of visualization like timeseries, pie chart, bar chart, histogram, XY chart, etc.
  - Integrates a wide range of data sources include Prometheus, MySQL, Jaeger, Loki, etc.
  - Can be configured to fire alerts when thresholds are breached
  - Create dynamic dashboards, integrating all data sources to provide a single holistic view of the system

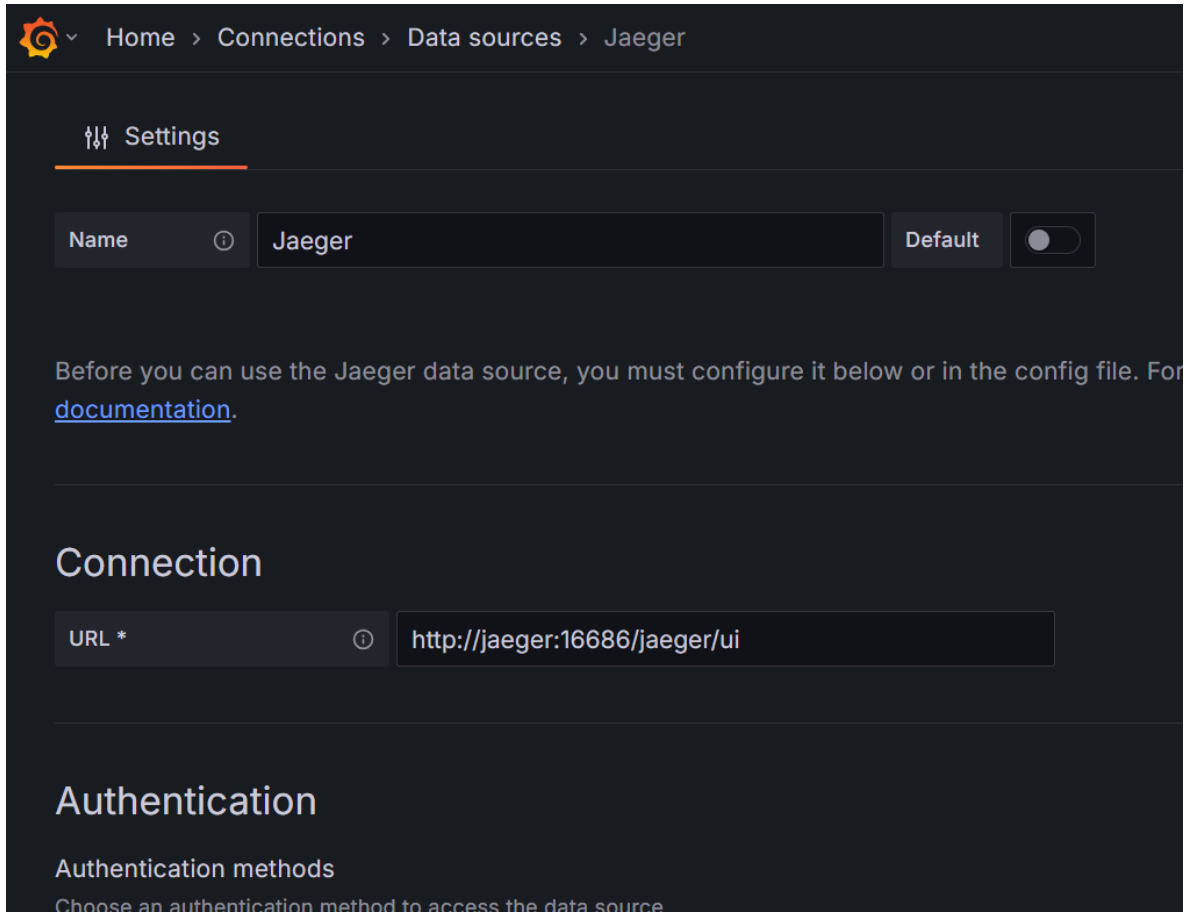
© Copyright National University of Singapore

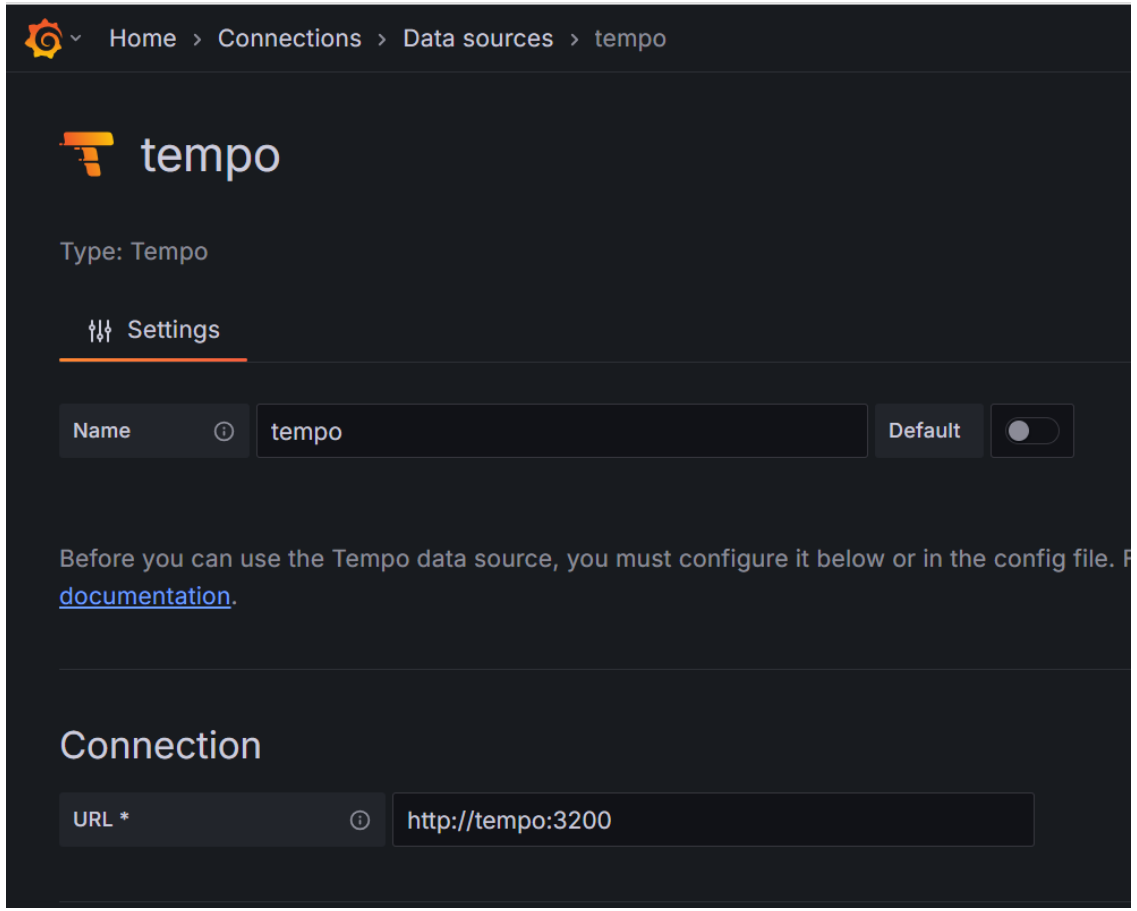
Grafana Data Sources



© Copyright National University of Singapore

Adding Data Sources to Grafana





`http://jaeger:16686/jaeger/ui`

`http://tempo:3200`

© Copyright National University of Singapore

Querying Traces

With TraceQL

© Copyright National University of Singapore

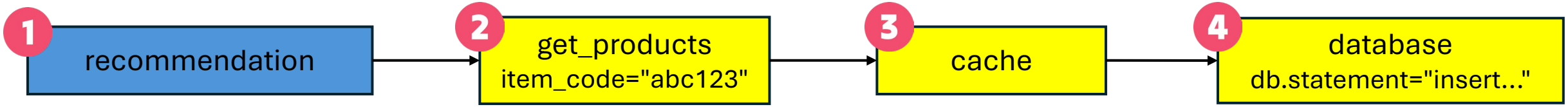
Building Blocks

- Attributes are encoded as key value pairs
  - Eg. **`service.name=hello`**, **`kind=SERVER`**, **`http.method=POST`**
- Scopes
  - **`resource`** is the entity that generated the entity
    - Eg. **`resource.service.name`**, **`resource.host.name`**
  - **`span`** details, all spans will have the same resource attribute if they are generated by that resource eg **`payment_service`**
    - Eg. **`span.http.method`**, **`span.user`**_id
  - **`event`** - span events (or annotations). Provides additional details about the operation
    - Must be manually added into the span
- Special attributes (intrinsics) that do not require scopes
  - Timing - **`duration`**
  - Span - **`name`**, **`kind`**, **`status`**
  - Trace - **`rootServiceName`**, **`duration`**
- TraceQL syntax
  - Only for Tempo data sources

```
{ attribute operator value }
```

© Copyright National University of Singapore

Basic Examples - 1



- Find all spans created by the recommendation service in all the traces

```
{ resource.service.name = "recommendation" }
```
1

- Find traces that query item `abc123`

```
{ resource.service.name = "get_products"
    && span.item_code = "abc123" }
```
2

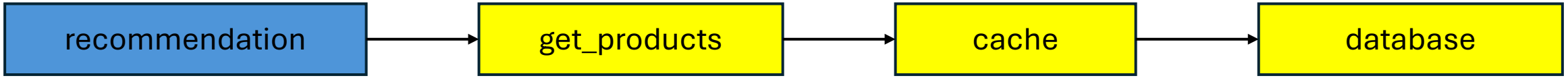
- Find all SQL inserts that took longer than 3 seconds

```
{ duration > 3s && span.db.statement =~ "(?i)insert.*"}
```
4

© Copyright National University of Singapore

`rootServiceName` vs `resource.service.name`

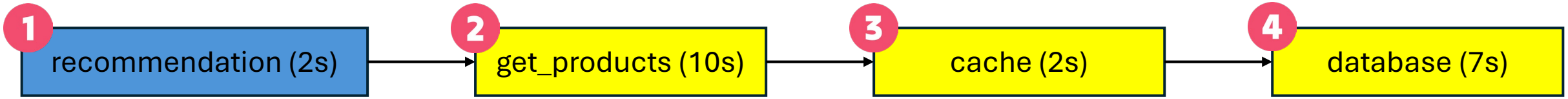
- **`rootServiceName`** is the service that initiated the entire trace
  - Trace level, filters all spans in the trace
  - Answers the question "How did this span start?"
- **`resource.service.name`** is the service that create the span
  - Span level, for filtering spans from a specific service
  - Answers the question "Who created this span?"



Span	rootServiceName	resource.service.name
recommendation	recommendation	recommendation
get_products	recommendation	get_products
cache	recommendation	cache
database	recommendation	database

© Copyright National University of Singapore

Basic Examples - 2



- Find all spans from any service that is initiated by recommendation service that took longer than 5 seconds

```
{ rootServiceName = "recommendation" && duration > 5s }
```
2 4

- Find any spans created by recommendation service that took longer than 5 seconds

```
{ resource.service.name = "recommendation" && duration > 5s }
```
2

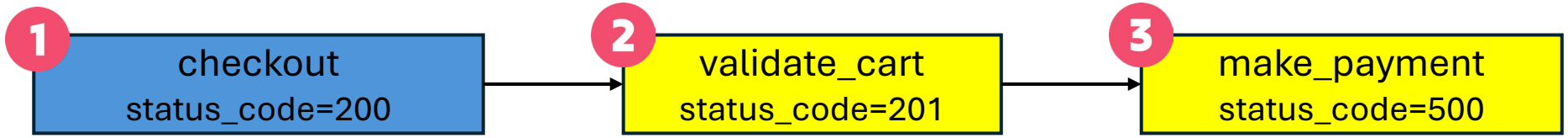
© Copyright National University of Singapore

Structural Queries

- Queries that explore the hierarchical relationships between a trace and its spans
- Operators
  - **>** - direct children only
  - **>>** - descendants
  - **~** - siblings
- Structural operators can be chained

© Copyright National University of Singapore

Structural Query Examples - 1



- Find any unsuccessful checkouts

```
{ rootServiceName = "checkout" }
    >> { span.http.status_code =~ "^[45][0-9][0-9]$" }
```

3

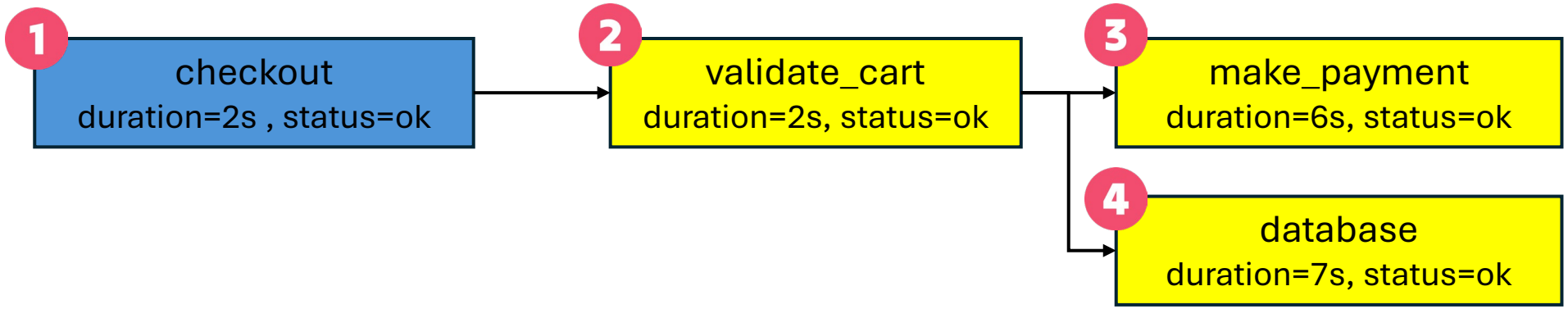
- Find any checkout spans that are unsuccessful

```
{ rootServiceName = "checkout"
    && span.http.status_code =~ "^[45][0-9][0-9]$" }
```

NOT FOUND

© Copyright National University of Singapore

Structural Query Examples - 2



- Find all successful checkouts that took longer than 5 seconds

```
{ rootServiceName = "checkout" }
    >> { duration > 5s }
    ~ { status = ok }
```

3 4

Aggregation

- Statistical function that compute summary traces
- Basic functions
  - Count operations
  - Duration aggregators eg avg, min, max, sum
  - Percentile functions for performance analysis
  - Histograms
- Aggregations can be group by specific criteria
  - Eg. **`http.status_code`**
- TraceQL queries with aggregation

```
{ attribute operator value } | aggregation_function
```

© Copyright National University of Singapore

Example - Basic aggregation

- Find all checkout traces where the total number of spans is greater than 3

```
{ resource.service.name = "checkout" } | count() >= 3
```

- Find all traces where the average duration is greater than 500ms

```
avg(duration) >= 500ms
```

- Find all spans with the **`response_size`** attribute that is greater than 1K
  - **`response_size`** type must be a number not a string, otherwise use regex

```
{ } | sum(span.response_size) >= 1000
```

© Copyright National University of Singapore

Example - Aggregation over Time - 1

- Checkout service error rate per second

```
{ resource.service.name = "checkout" && status = error } | rate()
```

- Calculate the status rate of spans from the checkout service

```
{ resource.service.name="checkout" } | rate() by (span.http.status_code)
```

- Count the status code from checkout service

```
{ resource.service.name="checkout" }
    | count_over_time() by (span.http.status_code)
```

© Copyright National University of Singapore

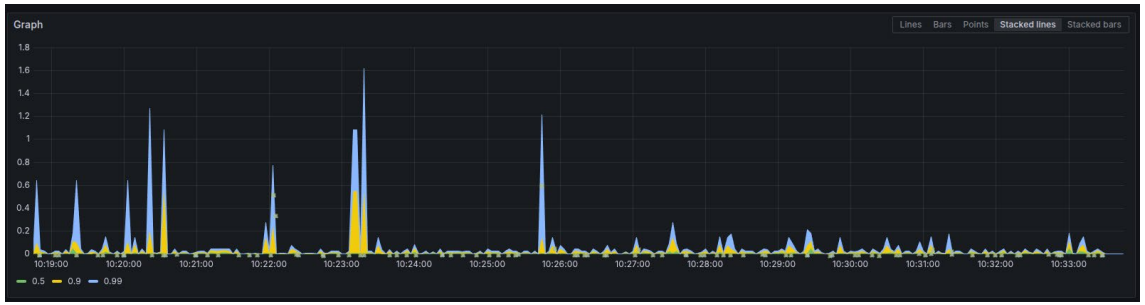
Example - Aggregation over Time - 2

- Latency distribution of the resource (URL) for checkout service

```
{ resource.service.name="checkout" }
    | histogram_over_time(duration) by (span.http.status_code)
```

- P99, P90 and P50 latency for checkout service

```
{ resource.service.name="checkout" }
    | quantile_over_time(duration, .99, .9, .5)
      by (span.http.status_code)
```



© Copyright National University of Singapore

Example - Top and Bottom K

- Top 5 most frequent error from checkout service

```
{ resource.service.name = "checkout" && status = error }
    | rate() by (span.http.route) | topk(5)
```

- 5 least used API from the checkout service

```
{ resource.service.name = "checkout" && span.http.route =~ "/api/.*" }
    | count_over_time() by (span.http.route) | bottomk(5)
```

© Copyright National University of Singapore

Workshop 1
Task 4

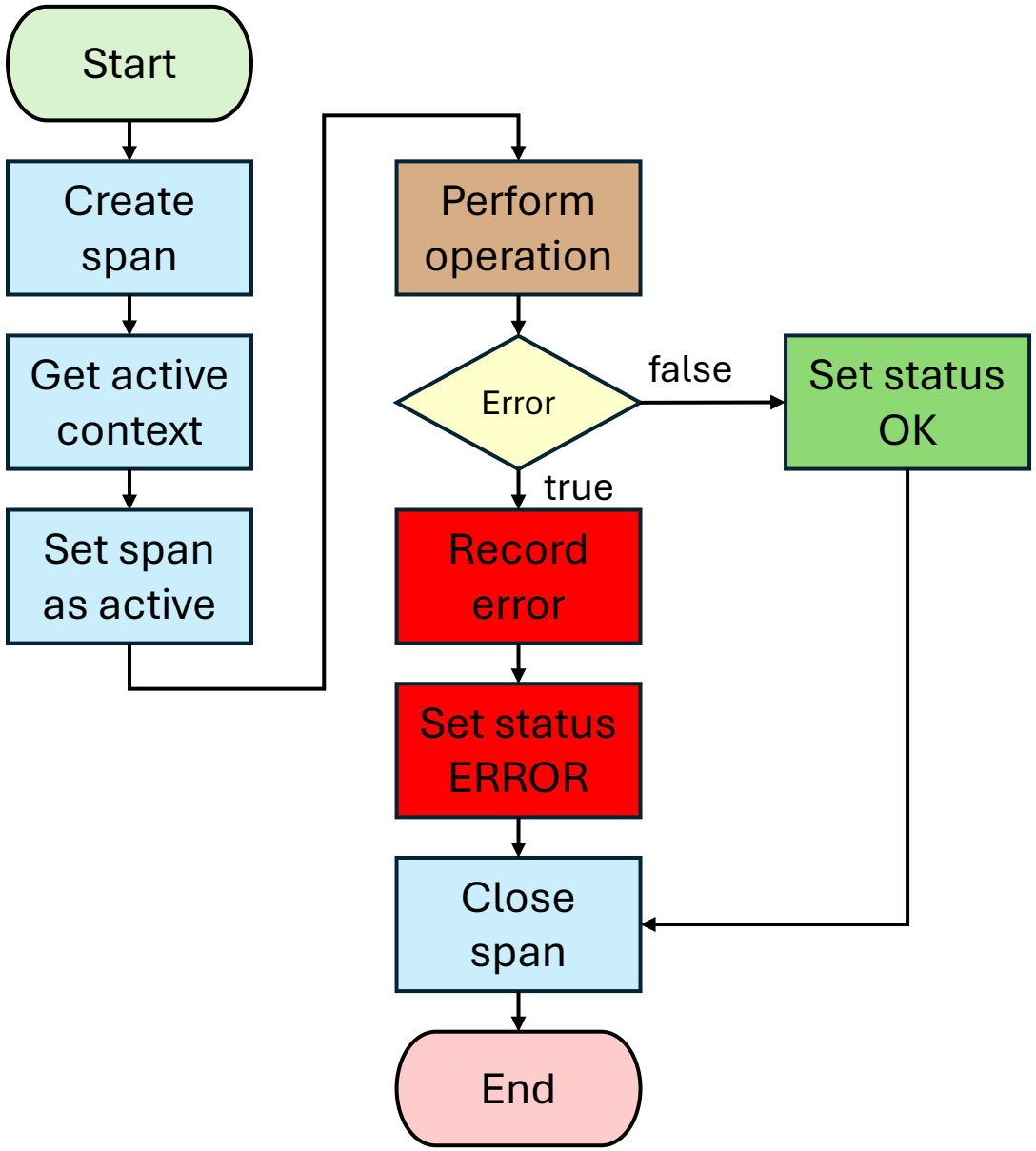
© Copyright National University of Singapore

Manual Trace Instrumentation

© Copyright National University of Singapore

Span and Context

- Create a span for an operation; make the span a child in the active context
  - Create a new span and sets its attributes
    - Eg. service name, span kind, etc
  - Get the current active context and make the span the active span
  - Perform operation
  - Set the span's status
    - OK if operation is successful
    - ERROR if operation fails
  - Close the span



© Copyright National University of Singapore

Example - Manual Context Management

get_weather: GET bd024ed

Find... | Trace Timeline

Trace Start **September 1 2025, 16:23:39.319** | Duration **67.52ms** | Services **1** | Depth **4** | Total Spans **5**

0µs | 16.88ms | 33.76ms | 50.64ms | 67.52ms

Service & Operation	0µs	16.88ms	33.76ms	50.64ms	67.52ms
get_weather GET					
get_weather request handler - /	63.29ms				
get_weather get_weather_from_openweathermap	62.07ms				
get_weather GET	34.45ms				
get_weather tcp.connect	21.12ms				

Custom span

Created by auto instrumentation under custom span

get_weather_from_openweathermap Service: **get_weather** | Duration: **62.07ms** | Start Time: **5ms**

Tags

city	hong kong
otel.scope.name	get_weather
otel.scope.version	1.0.0
service.name	get_weather
service.version	1.0.0
span.kind	client

Process: host.arch = amd64 | host.id = b93676df0cfa42289ae466b615b58c1b | host.name = batcomputer | process.command = /home/cmlee/tmp/study-span/main.js...

SpanID: e65093c9e8201e1e Copy

© Copyright National University of Singapore

Span Kind

- Describes the role of the work that the span represents
  - What kind of operation is this?
- Types of span kind
  - Client - make a call/request
    - Eg. HTTP request or a SQL query to a database
  - Server - receiving a request
    - Eg. web application request handler
  - Producer - sending a message
    - Eg. place a delivery order in a queue
  - Consumer - process a queued message
    - Eg. dequeuing a delivery order from a queue
  - Internal - internal service processing
    - Eg. making a function call, does not cross process boundary
- Kind is set when a span is created
  - Immutable, cannot be changed

© Copyright National University of Singapore

Status and Termination

- Set status to mark the operation as successful or failed
- Failed operation should also capture the failure details
  - Eg. exception trace
- Mark the operation complete by closing the span with **`end()`**
  - Allow the span duration to be calculated correctly
  - Trace and span duration can be used to derived latency metrics

© Copyright National University of Singapore

Example - Creating a Span

Create a new span in a new active context

Create a SERVER span

```
const tracer = trace.getTracer(appName, appVersion)

tracer.startActiveSpan('get_weather_from_openweathermap',
  { kind: SpanKind.SERVER }, async (span) => {

    span.setAttributes({
      [ ATTR_HTTP_REQUEST_METHOD ]: req.method, [ ATTR_URL_PATH ]: req.path
    })

    try {
      // Perform operation
      ...
      span.setStatus({ code: SpanStatusCode.OK }
    } catch (error) {
      span.recordException(error)
      span.setStatus({
        code: SpanStatusCode.ERROR,
        message: 'Operation failed'
      })
    } finally {
      span.end()
    }
  })
```

Set one or more span attributes

Work performed inside the span

Set the status of the span

Terminate the span when work completes

© Copyright National University of Singapore

Example - Span Error

get_weather: GET d0139a1

Find... | Trace Timeline

Trace Start **September 1 2025, 16:25:44.179** | Duration **1.97ms** | Services **1** | Depth **3** | Total Spans **3**

0µs | 491.25µs | 982.5µs | 1.47ms | 1.97ms

Service & Operation	0µs	491.25µs	982.5µs	1.47ms	1.97ms
get_weather GET				1.42ms	
get_weather request handler - /			965µs		
get_weather get_weather_from_openweathermap			613µs		

get_weather_from_openweathermap

Service: **get_weather** | Duration: **613µs** | Start Time: **1ms**

Tags: city = hong kong | otel.scope.name = get_weather | otel.scope.version = 1.0.0 | service.name = get_weather | service.version = 1.0.0 | span.kind = client

Process: host.arch = amd64 | host.id = b93676df0cfa42289ae466b615b58c1b | host.name = batcomputer | process.command = /home/cmlee/tmp/study-span/main.js...

Logs (1)

1.12ms

event	exception
exception.message	Invalid API key. Please consult documentation

Log timestamps are relative to the start time of the full trace.

SpanID: d36a589fdb806dbf Copy

© Copyright National University of Singapore

Parent-Child Span

1
```
tracer.startActiveSpan('get_weather_from_openweathermap', async (span) => {

  const result = getWeather(city)
  ...
  span.end()
})
```

Created inside **get_weather_from_openweather_map** span

```
function getWeather(city) {
```
2
```
  tracer.startActiveSpan('rest_request_to_openweather_map', async (span) => {

    const result = fetch('http://....')
    ...
    span.end()
  })
}
```

⚠️ Span kind defaults to INTERNAL if not specified

© Copyright National University of Singapore

Example - Parent-Child Span

get_weather: GET 093f10d

Find... | Trace Timeline

Trace Start **September 1 2025, 16:50:40.957** | Duration **8.08ms** | Services **1** | Depth **5** | Total Spans **5**

0µs | 2.02ms | 4.04ms | 6.06ms | 8.08ms

Service & Operation	0µs	2.02ms	4.04ms	6.06ms	8.08ms
get_weather GET					
get_weather request handler - /					7.6ms
get_weather get_weather_from_openweathermap					7.34ms
get_weather rest_request_to_openweather_map					6.77ms
get_weather GET					5.37ms

Child of

Child of

rest_request_to_openweather_map — Service: **get_weather** | Duration: **6.77ms** | Start Time: **0µs**

Tags: date = 1756716640957 | otel.scope.name = get_weather | otel.scope.version = 1.0.0 | service.name = get_weather | service.version = 1.0.0 | span.kind = server

Process: host.arch = amd64 | host.id = b93676df0cfa42289ae466b615b58c1b | host.name = batcomputer | process.command = /home/cmlee/tmp/study-span/main.js...

Logs (1)

18µs: event = fetch | http.method = GET | url = http://api.openweathermap.org/data/2.5/weather?appid=*REDACTED*&q=bangkok

Log timestamps are relative to the start time of the full trace.

SpanID: d823fd1973af35c7 Copy

© Copyright National University of Singapore

Sibling Span

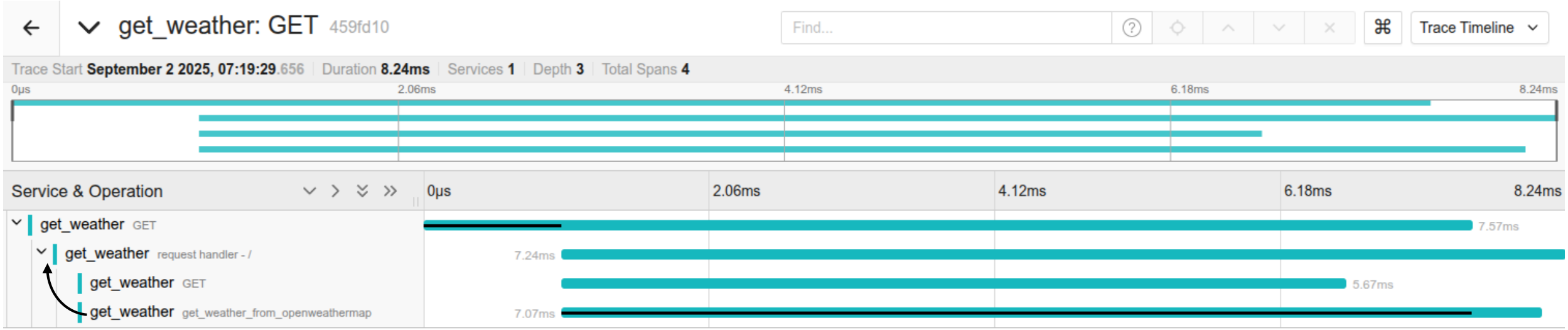
Get the current active context

```
const tracer = trace.getTracer(appName, appVersion)
const parentContext = context.active()

const span = tracer.startSpan('get_weather_from_openweathermap', {
  kind: SpanKind.SERVER,
  attributes: {
    [ ATTR_SERVICE_NAME ]: 'weather', [ ATTR_SERVICE_VERSION ]: 'v1.0.1'
  }
}, parentContext)

// Perform operation
...

span.setStatus(SpanStatusCode.OK)
span.end()
```

Create a new span in the current active context

© Copyright National University of Singapore

Example - Sibling Span



Child of the caller's active span

Example - Retrieving Active Span

- Get the current active span from OpenTelemetry
  - Useful when you only need to update the current span
- Need to check if active span is available
- Do not close the span
  - Let the span creator closes it

```
const { trace } = require('@opentelemetry/api')

// Operation
...

const span = trace.getActiveSpan()

if (!!span) {
  // Update span
  ...
  span.setStatus({
    status: SpanStatusCode.OK
  })
}
```

Get the current active span if available

Only update span if current active span is available

Workshop 1
Task 5 and 6

© Copyright National University of Singapore

Sampling

© Copyright National University of Singapore

Sampling

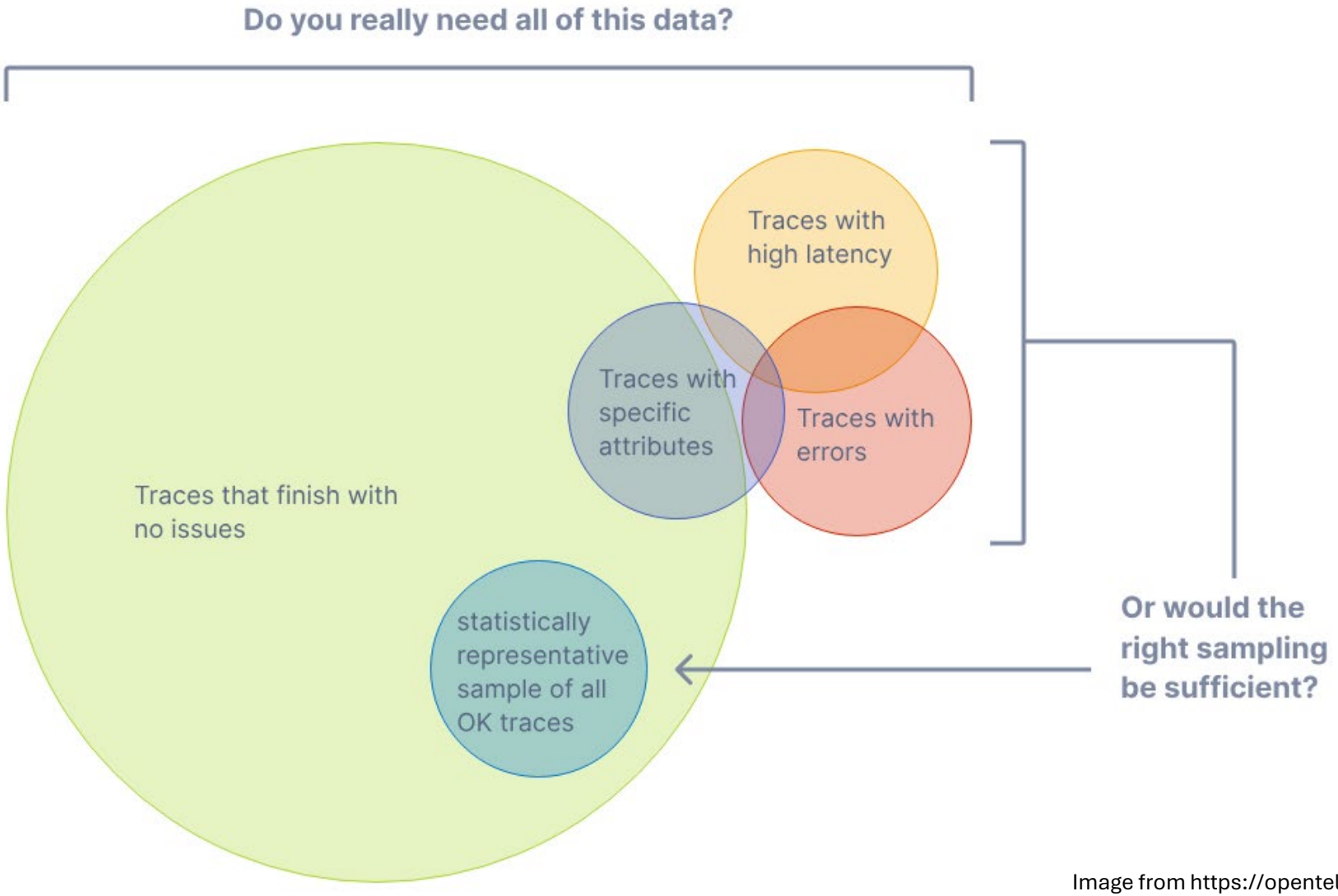


Image from https://opentelemetry.io/docs/concepts/sampling/

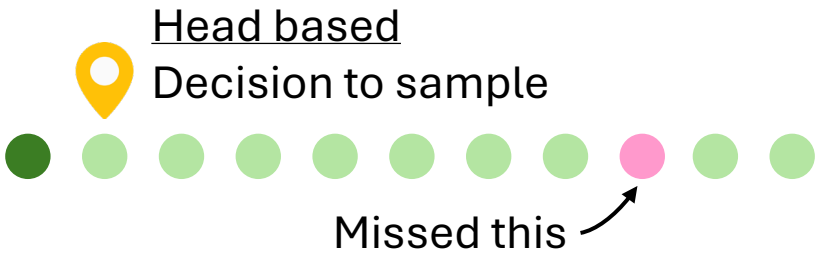
© Copyright National University of Singapore

Sampling

- Process of selectively collecting and storing only a subset of traces to forward to backends
  - Instead of capturing every single trace
- Goal is to able to capture enough to be able to understand latency issues, errors, trends, service interactions
  - Makes distributed tracing economically viable and operationally useful
- Benefits
  - Reduce storage cost
  - Too much data makes it harder to see meaningful patterns
  - Reduce network traffic and CPU utilization for transporting and processing traces
- Types of sample
  - Always-on/always-off - collect every trace
  - Probabilistic sampling - select traces based on probability eg 10%
  - Rule based sampling - select traces based on rule eg. always keep error traces
  - Head based sampling - select trace at when it starts
  - Tail based sampling - select trace when the trace completes

© Copyright National University of Singapore

Head and Tail Based Sampling



- Sampling decision made at the "head" of the trace
  - If sampled, then all downstream spans are sampled
  - If not sampled, then all downstream spans are dropped
  - Decision to sample or not to sample is propagated in the trace context
- Low overhead
- May miss important traces when event occur downstream

- Sampling decision made at the "tail" of the trace
  - Group all in spans relating to a trace for a short period of time eg 10 seconds
    - To allow the full trace to arrive
  - Decide if the trace should be kept/discarded based on predefined policies
- Intelligent policy based sampling
- High overhead for processing and buffering traces

Illustration adapted from https://logz.io/learn/sampling-in-distributed-tracing-guide/

© Copyright National University of Singapore

Example - Head-Based Sampling

Sample all traces in development but only 5% in production

```
let sampler = new AlwaysOnSampler()
if ('production' == process.env.NODE_ENV)
  sampler = new TraceIdRatioBasedSampler(.05)
const sdk = new NodeSDK({
  sampler,
  resource: resourceFromAttributes({
    [ATTR_SERVICE_NAME]: package.name,
    [ATTR_SERVICE_VERSION]: package.version
  }),
  spanProcessors: [ batchToOTLP, toConsole ],
  instrumentations: getNodeAutoInstrumentations()
})
...
```

Value between 0 and 1

- Head-based sampling is configured on the client
  - The application emitting the trace
- **`TraceIdRatioBasedSampler`** sample based on the configured threshold
  - Eg. Keep 5% of the traces, drop everything else

© Copyright National University of Singapore

ParentBasedSampler

- **`TraceIdRatioBasedSampler`** makes independent decision
  - Does not honor sampling decisions from upstream spans
  - Sampling information are stored in trace context
    - See https://www.w3.org/TR/trace-context/#version-format
- **`ParentBasedSampler`** makes sampling decision based on the entire trace
  - **`TraceIdRatioBasedSampler`** should be a component
  - Used only when the span is the root span

Follow remote spans sampling decision

Follow local spans sampling decision

```
const sampler = new ParentBasedSampler({
  root: new TraceIdRatioBasedSampler(0.2),
  remoteParentSampled: new AlwaysOnSampler(),
  remoteParentNotSampled: new AlwaysOffSampler(),
  localParentSampled: new AlwaysOnSampler(),
  localParentNotSampled: new AlwaysOffSampler()});
```

© Copyright National University of Singapore

Only Create Spans if Trace is Sampled

- Check if the trace is being sampled, especially in head-based sample
  - Avoid expensive work of creating spans

```
const currSpan = trace.getSpan(context.active())

if (!!currSpan && currSpan.isRecording()) {
  const tracer = trace.getTracer(appName, appVersion)
  tracer.startActiveSpan('get_weather_from_openweathermap', work)

} else
  work()
```

If the current active span is recording, then create spans for the trace

© Copyright National University of Singapore

Tail-Based Sampling

- Tail-based sampling is performed at the OTel collector
- Cannot be performed at the client
  - Individual service do not have access to spans from other services
  - Requires the complete trace
- Policy based decision instead of probabilistic
  - Eg. sample all error traces
  - Eg. sample all slow traces
- Tail sampling is configured as processor
  - Configure one or more policies
    - See https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/processor/tailsamplingprocessor
  - Add to pipelines to sample traces

© Copyright National University of Singapore

Example - Tail-Based Sampling

Configure **tail_sampling** processor

Wait 20s after the arrival of the first span of the trace before making sampling decision

Number of traces to store in memory

Policy name

Type of policy

Specific configuration

One or more sampling policies

```
processors:
  tail_sampling:
    decision_wait: 20s
    num_traces: 50000
    policies:
    - name: error_policy
      type: status_code
      status_code: {
        status_codes: [ ERROR ]
      }

service:
  pipelines:
    traces:
      receivers: [ otlp ]
      processors: [ tail_sampling ]
      exporters: [ tempo, jaeger ]
```

Include **tail_sampling** to pipelines

© Copyright National University of Singapore

Example - Sampling Policies - 1

Sample all traces with erroneous spans

```
name: error
type: status_code
status_code: {
  status_codes: [ ERROR ]
}
```

Sample all traces that are slower than 5s

```
name: slow_traces
type: latency
latency: {
  threshold_ms: 5000
}
```

Sample only **`checkout`** and **`add_inventory`** services

```
name: critical_services
type: string_attribute
string_attribute: {
  key: service.name
  values: [ checkout, add_inventory ]
}
```

Randomly sample 10% of the trace

```
name: random_10_percent
type: probabilistic
probablistic: {
  sampling_percentage: 10
}
```

© Copyright National University of Singapore

Example - Sampling Policies - 2

Sample all erroneous traces from **`checkout`** and **`add_inventory`** services

```
name: critical_services_error
type: and
and:
  and_sub_policy:
  - name: error
    type: status_code
    status_code: {
      status_codes: [ ERROR ]
    }
  - name: critical_services
    type: string_attribute
    string_attribute: {
      key: service.name
      values: [ checkout, add_inventory ]
    }
```

© Copyright National University of Singapore

End of Module

© Copyright National University of Singapore